

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE FUNDAMENTALS

Reactive System Architecture

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 10.4

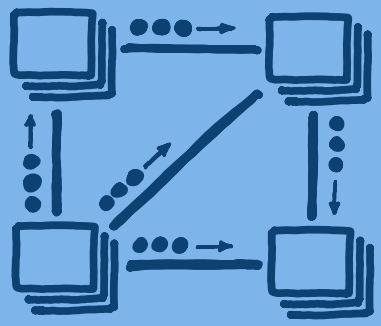
Intellectual Content: Version 1.11 (2020-11-09), Authored 2019-2020 by Michael Schäfer and Dr. Ralf S. Engelschall at rmsp Applied Technology Research
Graphical Illustration: Version 1.10 (2020-09-26), Copyright © 2019-2020 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Architecture & Systems

DEF Definition

Reactive System Architecture enables the realization of *Reactive Systems*.

Reactive Systems are in *subordinated interaction* with their *dominating environment*. They *continuously* process *endless data streams* as *small messages*, react at *any time* and respond within *tight time limits*. For this, they continuously *observe* their *environment* and adapt their *behaviour* to the current situation.



Demand & Deliverables

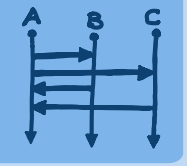
CTX Context

Real-time communication in the context of Digitization, Internet, Internet of Things (IoT), Systems of Engagement, Media and Analytics.



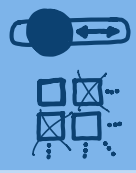
VAL Values

Non-blocking input/output data processing, *fast responses* within *tight time limits*, and *continuous availability* of the provided services.



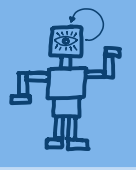
REQ Requirements

Services are *elastic* and provide high scalability, and are *resilient* and provide high fault tolerance.



PRP Properties

Services run *fully autonomously*, *monitor themselves*, and *automatically adapt* to changes in the environment.



Principles

Stay Responsive
Always respond in a timely manner.

Accept Uncertainty
Build reliability despite unreliable foundations.

Embrace Failure
Expect things to go wrong and design for resilience.

Assert Autonomy
Design components that act independently and interact collaboratively.

Tailor Consistency
Individualize consistency per component to balance availability and performance.

Decouple Time
Process asynchronously to avoid coordination and waiting.

Decouple Space
Create flexibility by embracing the network.

Handle Dynamics
Continuously adapt to varying demand and resources.

Patterns & Paradigms

ARC Architecture

Microservices, Cloud-Native Architecture (CNA), Event-Driven Architecture (EDA).



COM Communication

Asynchronous Communication, Non-Blocking I/O, Sequence, Push, Backpressure, Quality of Service (QoS).



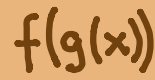
DAT Data

Semantical Event, Small Message, Endless Stream.



STY Style

Functional Programming, Asynchronous Programming.



\+

EXE Execution

Parallelization, Concurrency, Actors, Threads, Thread-Pool, Event-Loop.



INF Infrastructure

Message Queue (MQ), Load Balancer, Reverse Proxy, Service Mesh, Virtual Private Network (VPN).



PRC Processing

Complex Event Processing (CEP), EAI Patterns, Stream Processing (map, flatMap, filter, reduce), Event Sourcing.



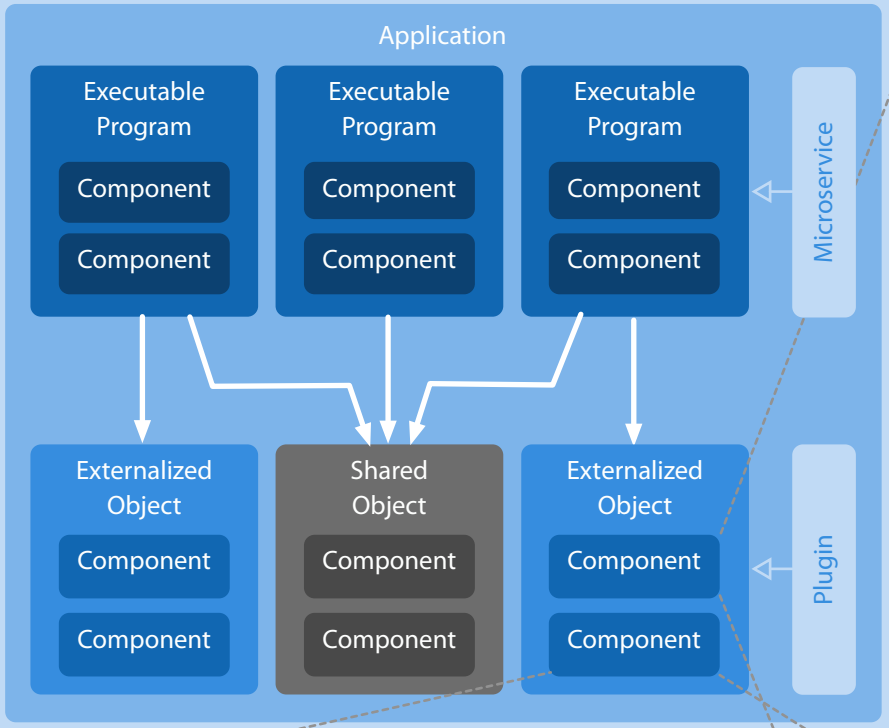
ASY Asynchronism

Callback, Promise/Future, Observable, Publish & Subscribe.

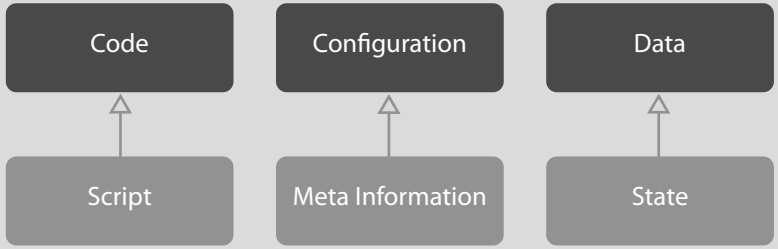


Application Composition

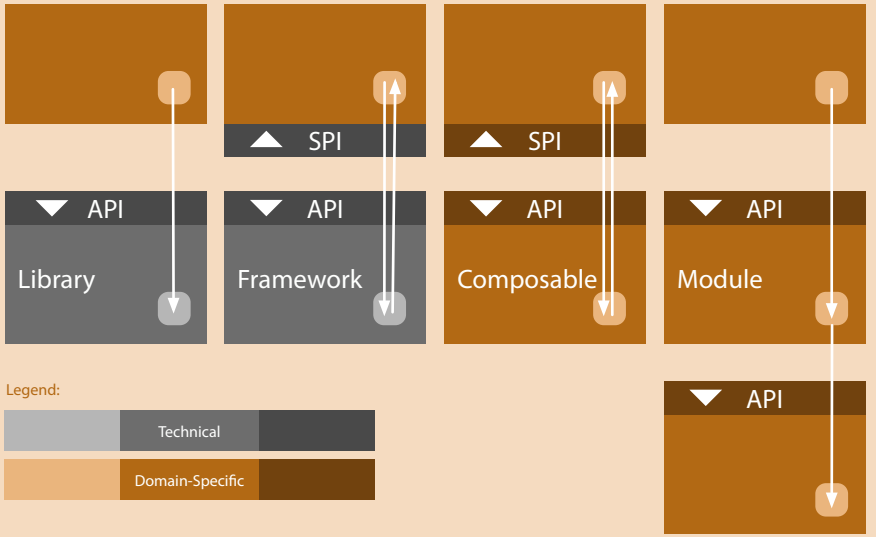
Application Structure



Component Assets



Component Types



Applications are composed out of one or more *Executable Programs*.

An *Executable Program* in turn can leverage zero or more (dedicated) *Externalised Objects* and/or (reusable) *Shared Objects*. Both *Executable Programs*, *Externalised Objects* and *Shared Objects* are composed through one or more *Components*.

Components are comprised of the assets *Code*, *Configuration* and *Data* and are of four distinct types: *Library*, *Framework*, *Composable* and *Module*, which are distinguished by their particular combination of characterizing aspects.

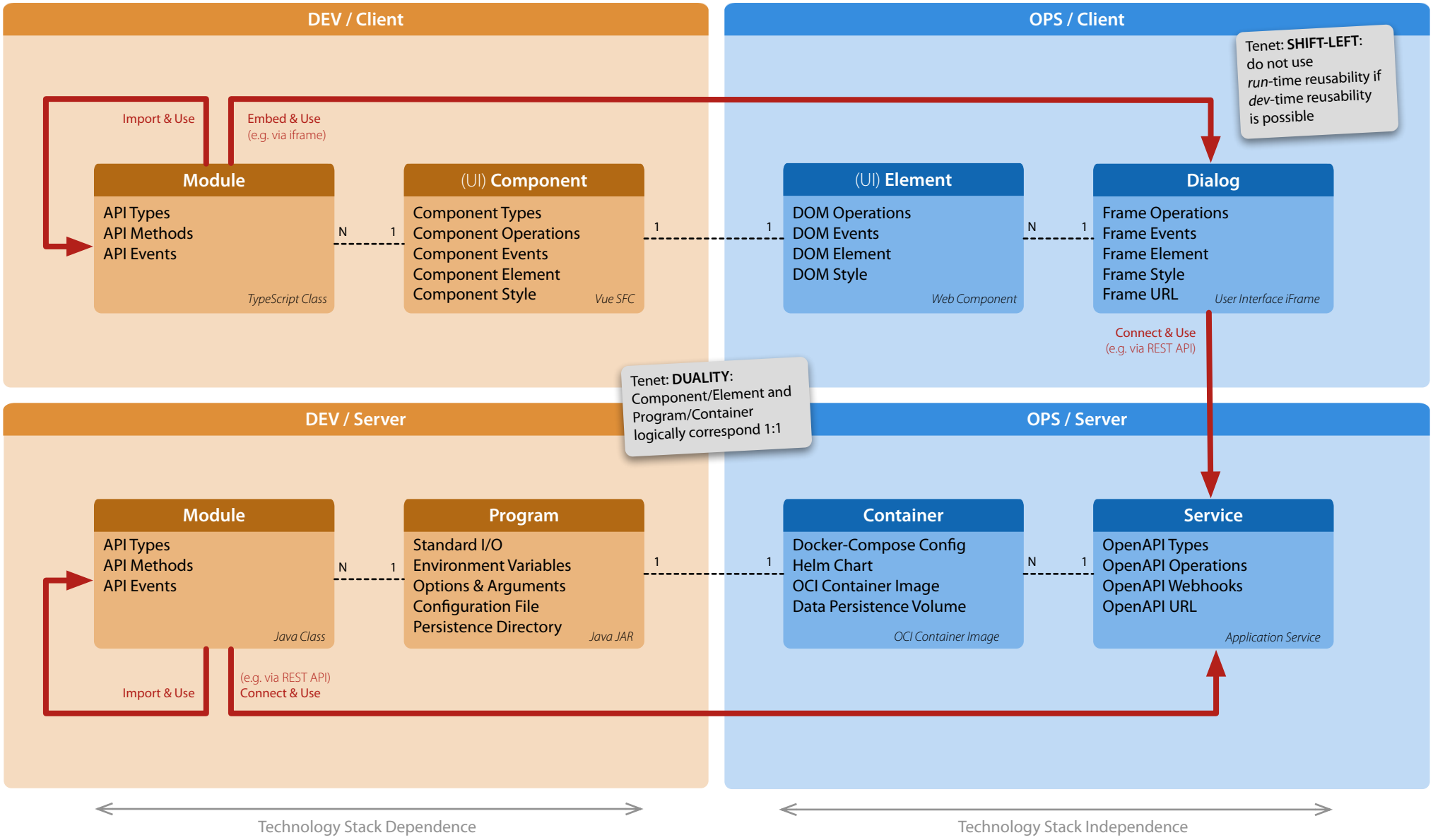
	data persistence	own life-cycle	requires integration	self-contained	invasive	passive	active	technical	domain-specific	reusable
Library				X		X		X		X
Framework		X	X	X	X	X	X	X		X
Composable	X	X	X	X			X		X	X
Module	X						X		X	

Intellectual Content: Version 1.02 (2022-05-24) Authored 2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.02 (2022-05-24) Copyright © 2022 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Functional Reusability

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 1.1.0 (2025-09-15), Authored 2024-2025 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.0 (2025-09-15), Copyright © 2024-2025 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Formal Languages

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 12.1

Declarative Languages

Express the target state
and let the machine figure out the steps.

Markup Languages

Write text intermixed with markup information.

```
foo <em>bar <strong>baz
</strong></em> quux
```

Examples:
Wiki, **Markdown**, AsciiDoc, SGML, **HTML**, TeX, R[un]off, reStructuredText, RTF

Configuration Languages

Express complex textual configurations.

```
foo bar quux { baz;
quux id 7; baz }
```

Examples:
INI, **XML**, SXML, **JSON**, **YAML**, TOML, HCL

Rule Languages

Express logic and semantic through complex rules.

```
foo(x, y) <- bar(x, y, z)
AND x < 42 AND z >= 10
```

Examples:
SQRL, Datalog/RuleML, OWL/**SWRL**, RIF

Constraint Languages

Find solutions for complex constraints.

```
foo @ bar(X, Y),
baz(X, Y, _) ==> quux.
```

Examples:
MiniZinc, CHR, **OCL**, Rego, Z3.

Query Languages

Retrieve information through paths and expressions.

```
// foo / bar [ @baz ==
"xxx" && @quux > 10 ]
```

Examples:
Glob, **RegExp**, **CSS Selector**, **XPath**, YARA, **GraphQL**, **SQL**, SPARQL, Cypher, GQL, **ASTq**

Validation Languages

Parse and validate complex textual information.

```
foo ::= "bar(#" (?:
[0-9a-fA-F]{2})+ ")"
```

Examples:
RegExp, Ducky, **BNF**, **PEG**, RELAX NG

solution approach: automatically, non-obvious
execution control: automatically, pre-defined
performance optimization: automatically, pre-defined

Imperative Languages

Express the steps
how the machine has to reach the target state.

Shell Languages

Automate execution of system commands.

```
foo -x 2>&1 | bar -y
--quux <(cat *.cf)
```

Examples:
Korn-Shell, Bourne-Shell, **Bash**, C-Shell, Batch-Script, **PowerShell**, AppleScript, DCL

Programming Languages

Execute complex algorithmic steps.

```
for (let i = 0; i < 10;
i++) foo(i, 42)
```

Examples:
JavaScript, **TypeScript**, **Scala**, **Kotlin**, Java, C#, C/C++, **Rust**, **Go**, Python, Perl, Ruby, Lua

Text-Processing Languages

Manipulate texts through transformations.

```
/^foo/,/bar.*baz/
s/quux\([0-9]*\)/foo\1/g
```

Examples:
ed, ex, **sed**, AWK, TXR, XSLT, JSLT

Macro Languages

Pre-process texts with macros.

```
define(`foo',`bar$1baz')
foo(quux)bar
```

Examples:
m4, GPP, CPP, **Zoem**, ProMac

Expression Languages

Expand path, arithmetic, and boolean expressions.

```
{{ foo.bar[*].baz[42]
.quux + 1 }}
```

Examples:
JQ, **YQ**, **MozJEXL**, MathML, JUEL, SpEL

Template Languages

Expand complex text fragments.

```
{% for k, v in items %}
{{k}}: {{v}}{% endfor %}
```

Examples:
Pug, **Nunjucks**, Handlebars, Mustache, Jinja, **Jsonnet**

solution approach: manually, obvious
execution control: manually, fine-grained
performance optimization: manually, fine-grained

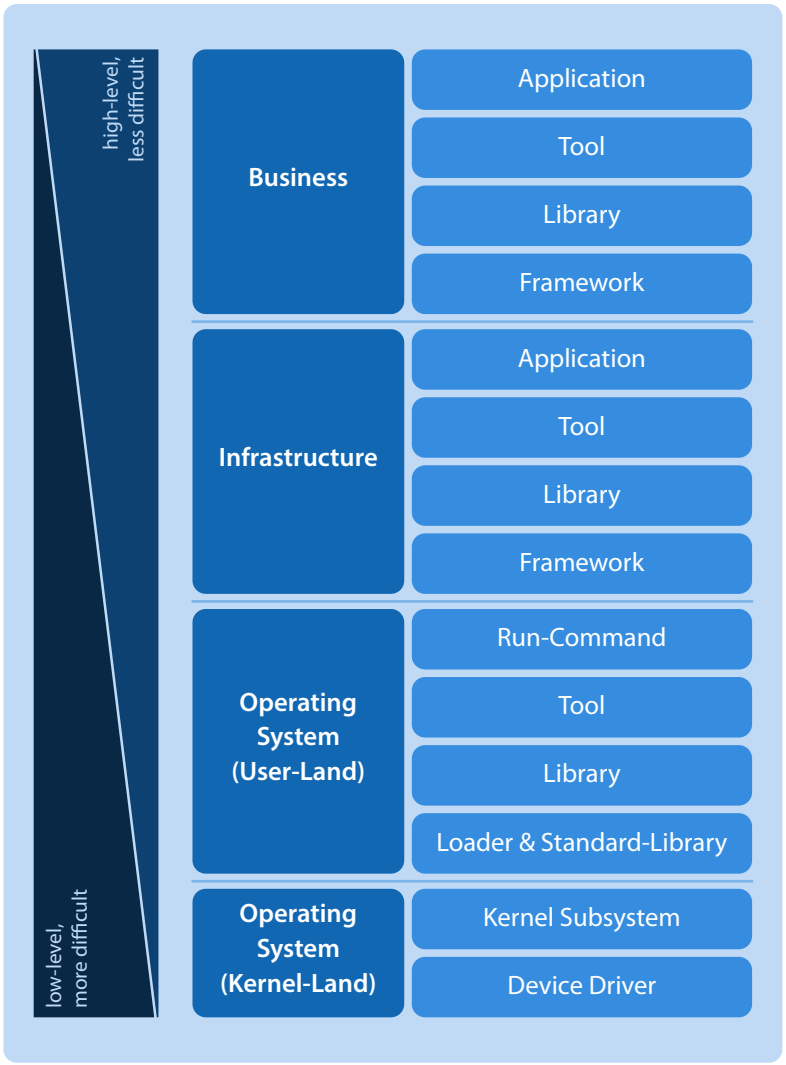
Examples:
essential
recommended
alternative

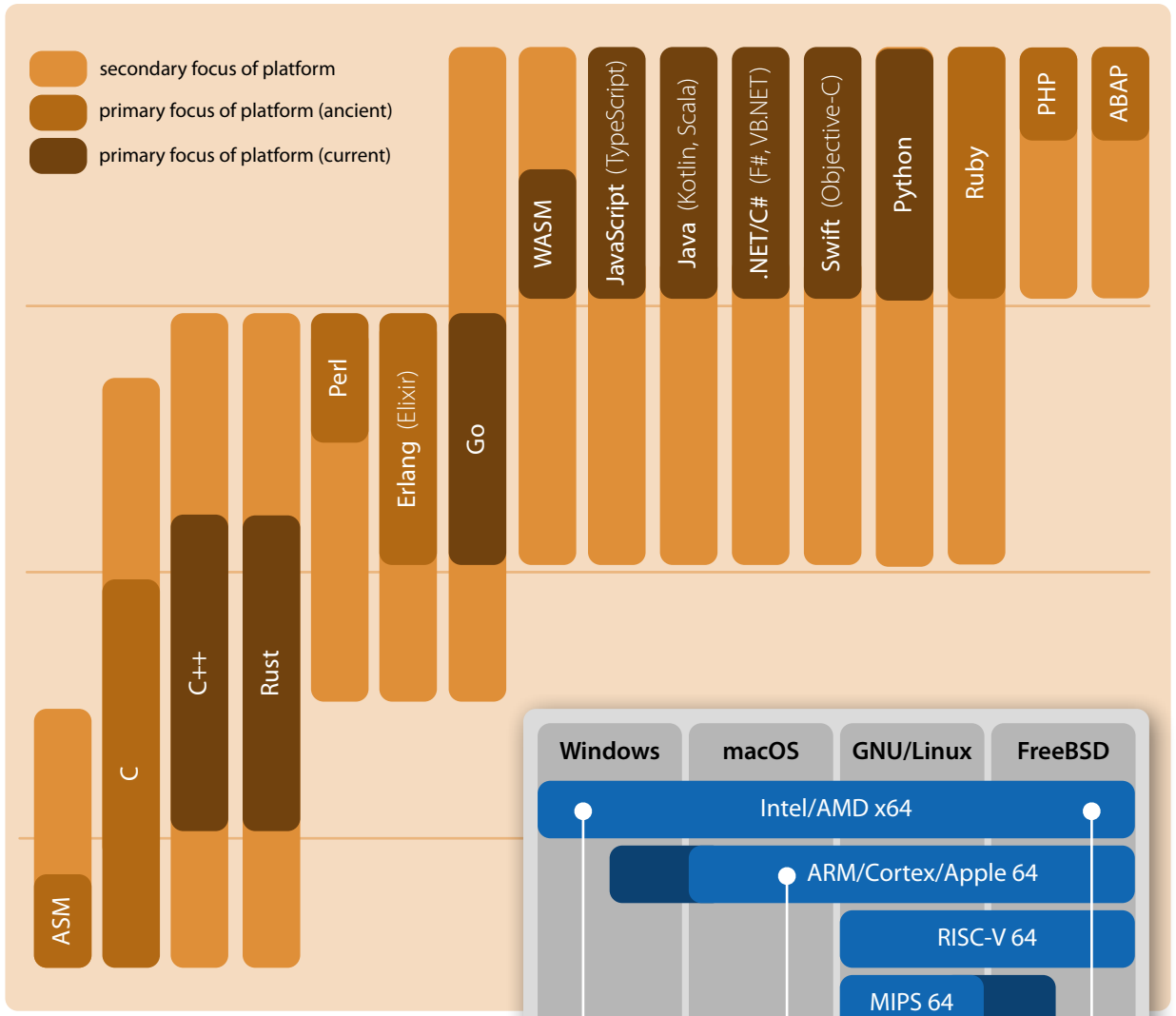
Intellectual Content: Version 1.0.5 (2020-10-17), Authored 2019-2020 by Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Graphical Illustration: Version 1.1.1 (2021-07-19), Copyright © 2019-2021 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

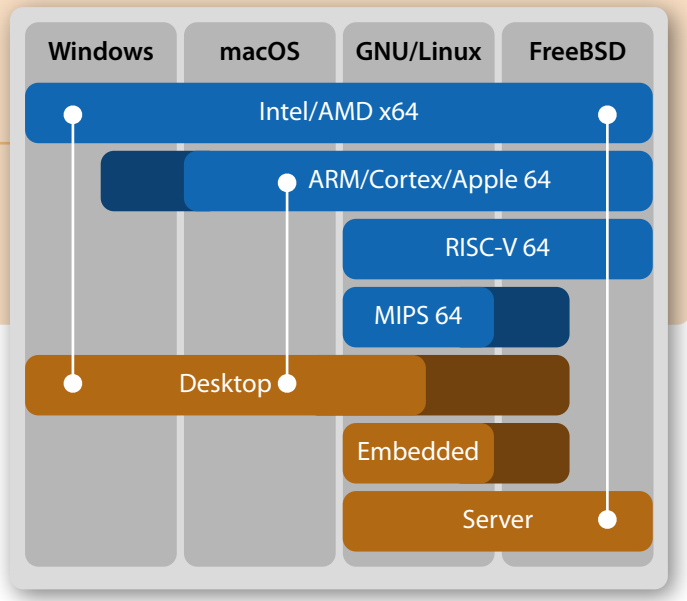
ARCHITECTURE FUNDAMENTALS

Technology Platforms

TECHNISCHE UNIVERSITÄT MÜNCHEN







Remember:
A *Technology Platform* is less about choosing a particular programming language and more about choosing a particular ecosystem for targeting a particular level of software!

Opinionated Recommendation (as of 2022):

Business:	**Scala, Kotlin, TypeScript, AssemblyScript**
Infrastructure:	**Go, Rust, Scala, Kotlin, TypeScript**
Operating System (UL):	**Rust, Go**
Operating System (KL):	**C, C++, Rust**

Typical Computing Devices (as of 2022):

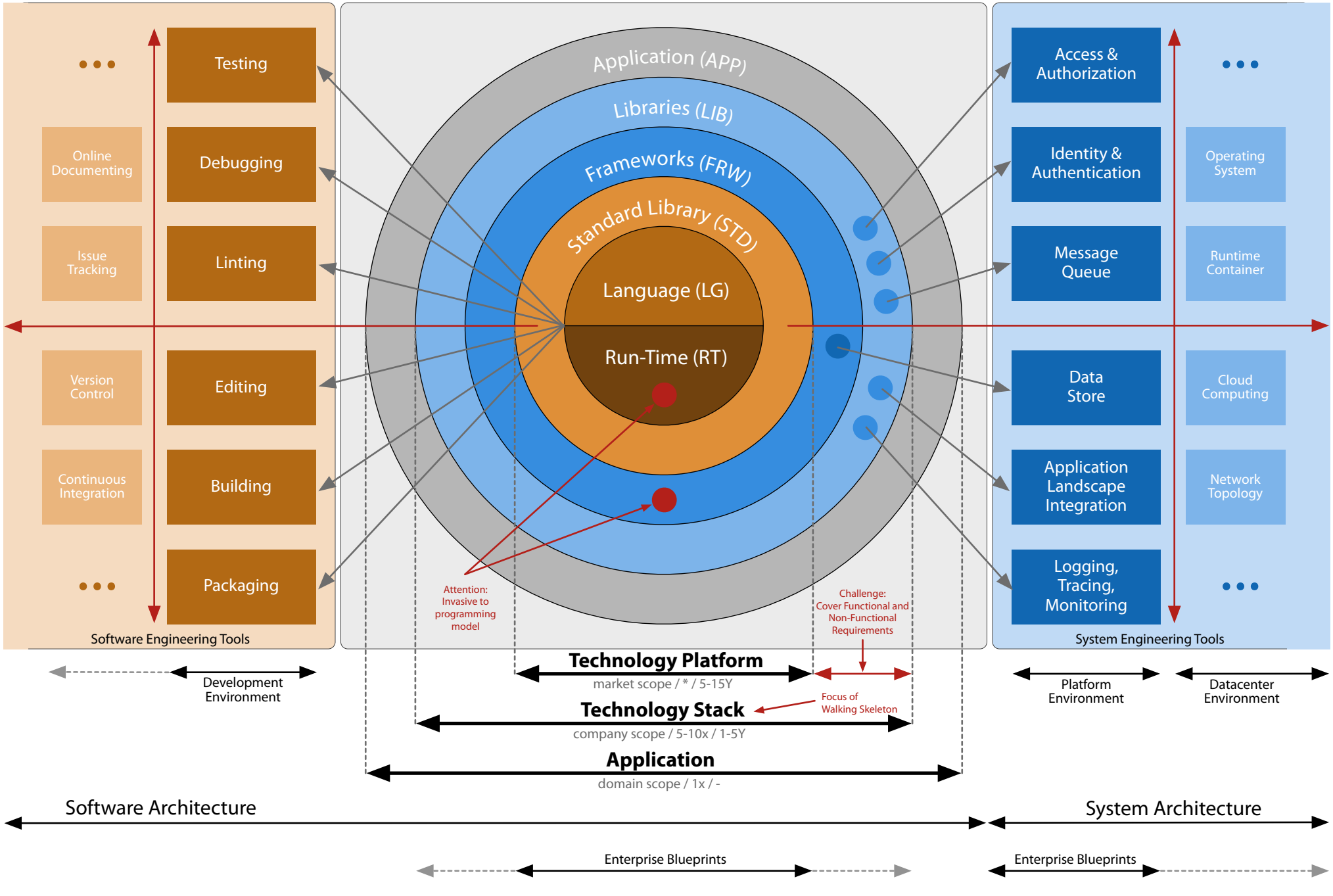
Intel/AMD x64:	**Personal Computer (PC)**
ARM/Cortex/Apple 64:	**Raspberry PI, BeagleBone, ROCKSPro64, iMac**
RISC-V 64:	**Beagle-V, HiFive Unmatched**
MIPS 64:	**Compex WPJ344**

Intellectual Content: Version 1.1.0 (2021-07-23), Authored 2020-2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.0 (2021-07-23), Copyright © 2020-2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Technology Stack

TECHNISCHE UNIVERSITÄT MÜNCHEN



AF 12.4

Intellectual Content: Version 1.05 (2019-10-27) Authored 2017-2019 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.06 (2021-10-05) Copyright © 2017-2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Rich-Client Aspects

TECHNISCHE UNIVERSITÄT MÜNCHEN

IT Interface Theme

Style Reset, Shape, Color, Gradient, Shadow, Font, Icon



Bootstrap | **TypoPRO, FontAwesome, Normalize**

IW Interface Widgets

Icon, Label, Text Paragraph, Image, Form, Text-Field, Text-Area, Date Picker, Toggle, Radio Button, Checkbox, Select List, Slider, Progress Bar, Hyperlink, Popup Menu, Dropdown Menu, Toolbar, Tooltip, Tab, Pill, Breadcrumb, Pagination, Badge, Alert, Panel, Modal, Table, Scrollbar, Carousel



Bootstrap | Select2, SlickGrid, ...

IL Interface Layouting

Responsive Design, Media Query, Frame, Grid, Padding, Border, Margin, Alignment, Force, Magnetism



Bootstrap | Swiper, jQuery Page, ...

IE Interface Effects

Transition, Transformation, Keyframes, Easing Function, Sound Effect, Physics



VueJS | **Animate.css,** DynamicJS, Howler, ...

II Interface Interactions

Mouse, Keyboard, Touchscreen, Gesture, Clipboard, Drag & Drop



VueJS | **Hammer, Mousetrap,** Dragula, ...

IS Interface States

Rendered, Enabled, Visible, Focused, Warning, Error, Floating



VueJS | (none)

IM Interface Mask

Markup Loading, Markup Generation, Virtual DOM, Text, Bitmaps, Vectors, 2D/3D Canvas, Accessibility



VueJS | **jQuery-Markup,** D3, Snap.svg, FabricJS, ...

18 Interface Internationalization

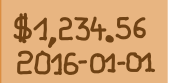
Text Internationalization (I18N).



VueJS | vue-i18next, I18Next

DC Data Conversion

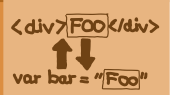
Value Formatting, Value Parsing, Localization (L10N).



VueJS | Moment, Numeral, Accounting, ...

DB Data Binding

Reactive, Observer, Unidirectional, Bidirectional, Incremental



VueJS | (none)

PM Presentation Model

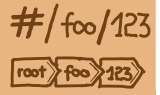
Parameter Value, Command Value, State Value, Data Value, Event Value, Value Validation, Presentation Logic



ComponentJS | (none)

DN Dialog Navigation

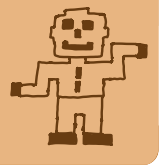
Deep Linking, Routing, Dialog Flow



ComponentJS | **Director,** URI.js

DA Dialog Automation

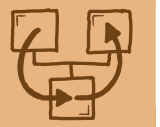
Dialog Macros, Click-Through, Smoke Testing.



ComponentJS | **ComponentJS-Testdrive**

DC Dialog Communication

Service, Event, Model, Socket, Hooks



ComponentJS | **Latching**

DL Dialog Life-Cycle

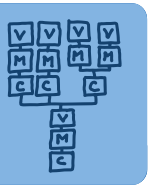
Component States, Component State Transitions.



ComponentJS | (none)

DS Dialog Structure

Component, Model/View/Controller Roles, Hierarchical Composition



ComponentJS | ComponentJS-MVC

SP State Persistence

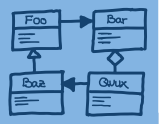
Local Storage, Cookies, Caching



(none) | Store.js, JS-Cookie

BM Business Model

Entity, Field, Relationship, Universally Unique Identifiers (UUID)



(none) | DataModelJS, Pure-UUID

UA Use-Case Authorization

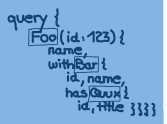
User Experience, Dialog Restriction, User, Group, Role, Use-Case, Data, Access.



(none) | (none)

CN Client Networking

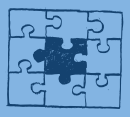
Request/Response, Synchronization, Push, Pull, Pulled-Push, REST, GraphQL, Authentication, Session.



(none) | **Axios,** Apollo Client

ED Environment Detection

Runtime Detection, Feature Detection.



(none) | Modernizr, FeatureJS, jQuery-Stage

AF 12.5

Intellectual Content: Version 1.1.5 (2020-07-24), Authored 2015-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.2.5 (2020-07-24), Copyright © 2015-2020 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Thin-Server Aspects

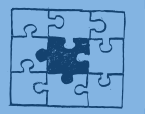
TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 12.6

ED Environment Detection

Detect the run-time environment, like underlying operating system, execution platform, network topology, feature toggles, etc.

Node | process, syspath



AP Argument Parsing

Parse options and arguments of the Command-Line Interface (CLI) to bootstrap application parameters.

(none) | yargs



CP Configuration Parsing

Load and parse directives from configuration file to bootstrap application parameters.

(none) | js-YAML



PD Process Daemonizing

Detdetach from the startup terminal and host process in order to run fully independently.

(none) | daemonize2



PM Process Management

(Pre-)fork child processes and/or threads of execution and monitor and control them during the life-cycle of the application.

(none) | cluster, nodemon



CM Component Management

Structure the code into components, instanciate them under run-time and manage them in a stateful component life-cycle.

Microkernel | (none)



CC Component Communication

Provide inter-component communication mechanisms like events, hooks, registry, etc.

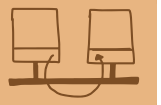
Microkernel | Latching



SN Server Networking

Listen to network sockets, accept connections and manage request/response and message communication.

HAPI | hapi-plugin-websocket, ws



PI Peer Information

Determine unique identification and add-on information about the client peer.

HAPI | hapi-plugin-peer, geoip



SH Session Handling

Manage secured per-connection sessions to keep state between communication requests and/or client sessions.

HAPI | YAR



UA User Authentication

Determine and validate the unique identity of the user communicating over the current network connection.

HAPI | JWT, Passport



RV Request Validation

Validate the syntactical and semantical compliance of the requests and sanitize the requests.

HAPI | Joi, DuckyJS



RP Request Processing

Process the request by dispatching execution according to the provided request and determined context information.

HAPI | GraphQL.js



RA Role Authorization

Determine whether the role of the current user is allowed to execute the current request.

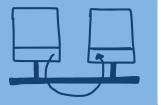
(none) | GraphQL-Tools-Sequelize



CN Client Networking

Provide mechanisms to connect to peers over the network and perform request/response and/or publish/subscribe communication.

(none) | Axios, MQTT.js, ws



TS Task Scheduling

Schedule and execute recurring tasks independent of regular I/O operations.

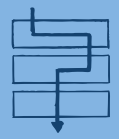
(none) | node-scheduler



ET Execution Tracing

Provide mechanisms for tracing the execution by logging event and measurement information at certain points of interest.

Microkernel | Winston



DA Database Access

Map in-memory domain entities onto data store dependent persistent data structure.

Sequelize | GraphQL-Tools-Sequelize



DC Database Connectivity

Locally or remotely connect the database access layer to the underlying data store.

Sequelize | sqlite3, pg



DS Database Schema

Create, update or downgrade the data schema inside the underlying data store.

Sequelize | (none)



DB Database Bootstrapping

Create, update or downgrade both mandatory bootstrapping and optional domain-specific data inside the underlying data store.

Sequelize | ini



Intellectual Content: Version 1.0.6 (2020-10-17), Authored 2017-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.3 (2018-10-25), Copyright © 2017-2018 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.